SCENE 01 / 18 · PART 1 · ALGORITHMIC · DANIEL · TARGET 0.3 MIN

STREAMING VIDEOS

STEP 1 OF 2

- Title. The 3D network turns slowly behind: data center at the back, three caches, two groups of users.
- “Hello, we are Group D. Our problem is Hash Code 2017, Streaming Videos.”

DANIEL · THE PROBLEM

0:00

OF 1:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Requests start flowing: every cube goes all the way to the data center and back.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

STREAMING VIDEOS

STEP 2 OF 2

- Requests start flowing: every cube goes all the way to the data center and back.
- "A video service keeps every video in one big data center, far from the users. Every request travels there and back, which is slow."
- "Near the users there are a few small cache servers. Our job is to decide which videos to copy into them."

SCENE 02 / 18 · PART 1 · ALGORITHMIC · DANIEL · TARGET 0.7 MIN

THE PROBLEM IN ONE MINUTE

STEP 1 OF 4

- The statement's own example: 5 videos, 3 caches of 100 MB, 2 endpoints.
- “This is the example from the statement. Endpoint 0 can reach three caches. Endpoint 1 has no cache at all.”

DANIEL · THE PROBLEM

0:00

OF 1:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Our choice + the rule. The request cards now show where each request is served.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 02 / 18 · PART 1 · ALGORITHMIC · DANIEL · TARGET 0.7 MIN

THE PROBLEM IN ONE MINUTE

STEP 2 OF 4

- Our choice + the rule. The request cards now show where each request is served.
- "We choose which videos go into which caches. A request is served by the fastest place that holds the video. Right now, that is always the data center."

DANIEL · THE PROBLEM

0:00

OF 1:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The statement's answer is placed; the score counts up to 462,500.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 02 / 18 · PART 1 · ALGORITHMIC · DANIEL · TARGET 0.7 MIN

THE PROBLEM IN ONE MINUTE

STEP 3 OF 4

- The statement's answer is placed; the score counts up to 462,500.
- "The score is the average time saved per request, in microseconds. The statement's own answer gets 462,500."

DANIEL · THE PROBLEM

0:00

OF 1:00

WHOLE TALK · TARGET 17:00

0:00

Next step
"Can we beat 462,500?"

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

THE PROBLEM IN ONE MINUTE

STEP 4 OF 4

- "Can we beat 462,500?"
- "Can we do better? Kacem will show how our algorithm works."
- HANDOVER to Kacem.

> *Handover to Kacem: "Can we do better? Kacem will show how our algorithm works."*

DANIEL · THE PROBLEM

0:00

OF 1:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

03 · WHY IT IS HARD, AND OUR PLAN

Kacem takes over

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 03 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 0.8 MIN

WHY IT IS HARD, AND OUR PLAN

STEP 1 OF 4

- Two reasons it is hard.
- "Two things make this hard. One: every cache is a knapsack, videos have sizes and the cache has a limit. Two: the caches interact. A copy is only worth the time it saves beyond what the users already get."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The scale from the statement.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 03 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 0.8 MIN

WHY IT IS HARD, AND OUR PLAN

STEP 2 OF 4

- The scale from the statement.
- "And the data sets go up to ten thousand videos, a thousand caches, a million request lines. Nothing is both exact and fast at this size."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The pipeline appears: floors 1 to 3 rise over the 3D network.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 03 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 0.8 MIN

WHY IT IS HARD, AND OUR PLAN

STEP 3 OF 4

- The pipeline appears: floors 1 to 3 rise over the 3D network.
- "So we stack methods. Read and index the file. Build a fast answer with a greedy. Then improve each cache exactly."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Floors 4 and 5, and the rule that ties them together.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

WHY IT IS HARD, AND OUR PLAN

STEP 4 OF 4

- Floors 4 and 5, and the rule that ties them together.
- "Then change two caches together, and for small data sets solve the whole problem exactly."
- "Each floor starts from the answer below it and only keeps a change if the score does not drop."

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 1 OF 8

- The queue: every (cache, video) pair with its saving and saving per MB.
- "Here is the greedy, step by step, on the statement's example. First we score every pair: requests times the time saved. For cache 0 and video 3, that is 1,500 times 900 milliseconds, divided by its 30 MB: 45,000 per MB."
- "Video 4 is 110 MB, it never fits. Endpoint 1 has no cache, so it never enters the queue."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Pop 1: cache 0 · video 3 → place.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 2 OF 8

- Pop 1: cache 0 · video 3 → place.
- "We always take the best pair, re-check its saving, and if it has not changed we place it. Video 3 goes into cache 0."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Pop 2: cache 2 · video 3 → drop (the key moment).

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 3 OF 8

- Pop 2: cache 2 · video 3 → drop (the key moment).
- "Next is video 3 in cache 2. It looked worth 40,000 per MB. But endpoint 0 now gets video 3 from cache 0 in 100 ms. Re-checked, it saves nothing. We drop it. This re-check is what stops us storing useless copies."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Pop 3: cache 1 · video 3 → drop.

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 4 OF 8

- Pop 3: cache 1 · video 3 → drop.
- "Same for cache 1."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Pop 4: cache 0 · video 1 → place. Score 562,500.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 5 OF 8

- Pop 4: cache 0 · video 1 → place. Score 562,500.
- "Video 1 into cache 0: unchanged, placed. Cache 0 is now 80 out of 100 MB."

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 6 OF 8

- Pops 5 and 6 dropped.
- "Video 1 in the other caches: already served by cache 0. Dropped."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Round over: evict and swap find nothing. +100,000 vs the statement.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 7 OF 8

- Round over: evict and swap find nothing. +100,000 vs the statement.
- "After the round we evict useless copies and try swaps. Nothing to do here. We stop with 562,500: 100,000 more than the statement's answer."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Is it optimal? Yes, and the exact solver agrees.

RUNNING IT ON THE EXAMPLE, POP BY POP

STEP 8 OF 8

- Is it optimal? Yes, and the exact solver agrees.
- "And here it is the best possible: endpoint 0 gets both videos from its fastest cache, and nothing else can be improved. Our exact solver proves it."

SCENE 05 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 1.2 MIN

WHEN "BEST PER MB" IS FOOLED

STEP 1 OF 5

- Evict and swap, after every round.
- “Two clean-ups run after every round. Evict removes copies that no longer save anything. Swap trades the weakest copy for a better video.”

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The ratio trap: one cache of 10 MB.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 05 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 1.2 MIN

WHEN "BEST PER MB" IS FOOLED

STEP 2 OF 5

- The ratio trap: one cache of 10 MB.
- “But choosing by saving per MB can be fooled. One cache of 10 MB. Video A has the best ratio, so the greedy takes it. Then nothing else fits: 600, and 4 MB wasted.”

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step
The DP table fills.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

WHEN "BEST PER MB" IS FOOLED

STEP 3 OF 5

- The DP table fills.
- "The fix: freeze all the other caches. Then each video is worth a fixed amount in this cache, and filling it is exactly a knapsack. We solve it optimally with dynamic programming."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The path back through the table: B + C = 900.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

WHEN "BEST PER MB" IS FOOLED

STEP 4 OF 5

- The path back through the table: B + C = 900.
- "The table says: take B and C. 900 instead of 600."

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Repeat cache after cache. The slider lets you try other sizes.

SCENE 05 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 1.2 MIN

WHEN "BEST PER MB" IS FOOLED

STEP 5 OF 5

- Repeat cache after cache. The slider lets you try other sizes.
- "We repeat this for every cache until nothing changes. It can only go up, so it always stops."
- If asked: move the slider. At 11 MB both methods agree.

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

06 · TWO CACHES AT ONCE, THEN EXACT

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 06 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 1.3 MIN

TWO CACHES AT ONCE, THEN EXACT

STEP 1 OF 5

- The trap instance after the greedy: B gets nothing, ~50% of the optimum.
- “But re-pack changes one cache at a time. Here is a trap from our generator. Endpoint A's videos look slightly better in cache a, so the greedy puts them there. Endpoint B gets nothing: half the optimum.”

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Re-pack is stuck.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

TWO CACHES AT ONCE, THEN EXACT

STEP 2 OF 5

- Re-pack is stuck.
- “Re-pack cache a alone: A's videos are still the best there. Re-pack cache b alone: A already gets them from a, so they are worth zero in b. Each cache is optimal given the other. Stuck.”

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

LNS: both caches emptied and rebuilt together. Score jumps to 998,499.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

TWO CACHES AT ONCE, THEN EXACT

STEP 3 OF 5

- LNS: both caches emptied and rebuilt together. Score jumps to 998,499.
- “Large neighbourhood search empties two caches that share users and rebuilds them together. Now A goes to b and B goes to a. We keep a change only if the score does not drop.”

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The exact solver: the integer program and the real output on this instance.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 06 / 18 · PART 1 · ALGORITHMIC · KACEM · TARGET 1.3 MIN

TWO CACHES AT ONCE, THEN EXACT

STEP 4 OF 5

- The exact solver: the integer program and the real output on this instance.
- “For small data sets we also write the whole problem as an integer program and solve it exactly. It proves 998,499 is optimal here, and the zoo data set in 3 seconds. On big ones it still gives a ceiling: on videos, no answer can pass 620,009.”

KACEM · THE ALGORITHM, STEP BY STEP

0:00

OF 5:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The ladder, computed live. The class trap data sets went from 50–67% to 100%.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

TWO CACHES AT ONCE, THEN EXACT

STEP 5 OF 5

- The ladder, computed live. The class trap data sets went from 50–67% to 100%.
- "Live, on this instance: greedy, re-pack, LNS, exact. And the same kind of trap in the class sheet: from about half the optimum to proven optimal."
- HANDOVER to Dhia.

> *Handover to Dhia: "Now, how well does it work? Dhia."*

EXPERIMENTAL PROTOCOL

STEP 1 OF 4

- Data sets.
- “How we measured. Four Google data sets, the 29 class data sets, the 15 of the local contest, and 12 we generated ourselves. Before anything, our scorer had to reproduce the statement's 462,500.”

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Settings.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

EXPERIMENTAL PROTOCOL

STEP 2 OF 4

- Settings.
- "Same laptop for every method. Greedy up to five rounds; re-pack on the best 500 candidates per cache; LNS on two caches, five minutes per data set; the exact solver only on small models, 120 seconds."

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Limits.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

EXPERIMENTAL PROTOCOL

STEP 3 OF 4

- Limits.
- "The limits: a time budget per data set, runtimes from one laptop only, one LNS run per data set, and the class best is the sheet on 31 October."

EXPERIMENTAL PROTOCOL

STEP 4 OF 4

- Metrics.
- "The official score; our score as a share of the best any group reached; the professor's three rankings; and proof: a proven optimum or the gap to a proven ceiling. Every output is re-scored independently."

THE OFFICIAL RESULT: LOCAL CONTEST

STEP 1 OF 4

- Totals as a share of the best possible: L, F, then us.
- "The official result: the local contest. We finished third of twelve, by all three rankings. 2,870 points behind first, out of almost 23 million."

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Heat map: data set × team. Our column is framed.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

THE OFFICIAL RESULT: LOCAL CONTEST

STEP 2 OF 4

- Heat map: data set × team. Our column is framed.
- "Every cell is one team on one data set, as a share of the best. Our worst data set is still above 99.3% of the best. Most teams fall below 90% somewhere."

THE OFFICIAL RESULT: LOCAL CONTEST

STEP 3 OF 4

- Our best cells only.
- "We have the best score on 9 of the 15 data sets, and we are the only team at the best on one of them."

SCENE 08 / 18 · PART 2 · EXPERIMENTAL EVALUATION · DHIA · TARGET 1.5 MIN

THE OFFICIAL RESULT: LOCAL CONTEST

STEP 4 OF 4

- First to second submission.
- “Between our two submissions we gained 6,800 points. We were already third with the first.”

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

09 · THE CLASS BENCHMARK: FROM 6TH TO 2ND

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

THE CLASS BENCHMARK: FROM 6TH TO 2ND

STEP 1 OF 4

- V1 → V3 on the class data sets.
- "On the class benchmark, the greedy versions first: cache by cache, then eviction, then one global heap with swaps. Plus 11.7 percent."

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next step
The full pipeline: 6th → 2nd.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

THE CLASS BENCHMARK: FROM 6TH TO 2ND

STEP 2 OF 4

- The full pipeline: 6th → 2nd.
- "Then the full pipeline: plus 1.87 million points on the 29 class data sets, from sixth to second in the class sheet."

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Where the points came from: the traps.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

THE CLASS BENCHMARK: FROM 6TH TO 2ND

STEP 3 OF 4

- Where the points came from: the traps.
- "Most of it comes from the trap data sets: the greedy was at 50 to 67 percent, now they are proven optimal. Eight data sets are proven optimal in total."

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Google data sets, and the ceiling on videos.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

THE CLASS BENCHMARK: FROM 6TH TO 2ND

STEP 4 OF 4

- Google data sets, and the ceiling on videos.
- "On the four Google data sets: zoo is proven optimal, and on videos we know we are at most 6,158 points from perfect."
- HANDOVER to Marouane.

> *Handover to Marouane: "Marouane will explain how we chose each setting."*

DHIA · PROTOCOL AND RESULTS

0:00

OF 4:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

10 · EVERY CHOICE WAS MEASURED

Marouane takes over

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

EVERY CHOICE WAS MEASURED

STEP 1 OF 3

- LNS vs annealing.
- "Every parameter was chosen by a test. We implemented simulated annealing and LNS and compared them from the same start, same time. LNS gained more and won on six of nine. Annealing needs the right temperature, and it changes per data set."

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Group size and time.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

EVERY CHOICE WAS MEASURED

STEP 2 OF 3

- Group size and time.
- "Two caches per LNS step beats four or eight: smaller groups rebuild faster. And gains flatten after a few minutes, so five minutes per data set."

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Exact-solver threshold.

EVERY CHOICE WAS MEASURED

STEP 3 OF 3

- Exact-solver threshold.
- "The exact solver wins below ten thousand variables and loses above two hundred thousand, so we set the limit at fifty thousand."

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

11 · SWOT

SWOT

STEP 1 OF 4

- Strengths.
- “Strengths: no step can lower the score, eight proven optima, and consistency: never below 99.3% of the best in the contest.”

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Weaknesses.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SWOT

STEP 2 OF 4

- Weaknesses.
- "Weaknesses: the greedy alone falls into traps, our improvements are local, and the exact solver stops at fifty thousand variables."

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Opportunities.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SWOT

STEP 3 OF 4

- Opportunities.
- "Opportunities: about six thousand points left on videos, parallel LNS, and splitting big integer programs."

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step
Threats.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SWOT

STEP 4 OF 4

- Threats.
- "Threats: the top three of the contest are within a hundredth of a percent, so small details decide the rank."

MAROUANE · MEASURED CHOICES AND SWOT

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

12 · HOW THE FINAL SUBMISSION IS BUILT

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 12 / 18 · PART 3 · CODE · MAROUANE · TARGET 1.5 MIN

HOW THE FINAL SUBMISSION IS BUILT

STEP 1 OF 3

- Core flow: .in → solve.py → solver.cpp.
- “Now the code of our final submission. One command, solve.py, takes a data set and a time budget, and runs our C++ solver: greedy, re-pack, LNS.”

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Exact solver in parallel, re-score, keep the best.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

HOW THE FINAL SUBMISSION IS BUILT

STEP 2 OF 3

- Exact solver in parallel, re-score, keep the best.
- “If the model is small, it runs the exact solver at the same time. Every candidate is re-scored by our independent Python scorer and checked for capacity. The best valid one is the .out.”

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The tools around it. Click any box to show what it does.

HOW THE FINAL SUBMISSION IS BUILT

STEP 3 OF 3

- The tools around it. Click any box to show what it does.
- "Around it: the instance generator, a local runner page, and the class console you used, which reuses our parser and scorer."

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

13 · IMPLEMENTATION CHOICES

IMPLEMENTATION CHOICES

STEP 1 OF 4

- Python first, then C++ · real demand only.
- "Two choices for speed. We wrote the Python version first and ported it move by move to C++: same scores, kittens from six minutes to 13 seconds. And a cache only ever looks at the videos its own users request."

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Lazy heap · best and second-best latency.

IMPLEMENTATION CHOICES

STEP 2 OF 4

- Lazy heap · best and second-best latency.
- “The heap is lazy: gains only go down, so we only re-check the top. And we store the best and second-best latency per request, so re-pack gets "fastest from another cache" instantly.”

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Bitset knapsack · always a valid answer.

IMPLEMENTATION CHOICES

STEP 3 OF 4

- Bitset knapsack · always a valid answer.
- "The knapsack uses one bit per cell, and dividing sizes by their gcd makes it up to 5,000 times faster on equal sizes. And the first greedy round always finishes, so there is always a valid answer on time."

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Trust nothing · last round of tuning.

IMPLEMENTATION CHOICES

STEP 4 OF 4

- Trust nothing · last round of tuning.
- "Nothing is submitted before an independent re-score. And with the second contest submission we added extra swap and replace moves between neighbouring caches."
- HANDOVER to Daniel.

> *Handover to Daniel: "Daniel will conclude."*

MAROUANE · THE FINAL SUBMISSION

0:00

OF 3:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

14 · SINCE OUR LAST PRESENTATION

Daniel takes over

SCENE 14 / 18 · PART 4 · CONCLUSION · DANIEL · TARGET 1 MIN

SINCE OUR LAST PRESENTATION

STEP 1 OF 5

- Last time: 4th of 9, Python only. The list of next steps we promised.
- “To conclude, let's go back to our last presentation. We were fourth of nine, with a Python greedy, and we listed what we would do next.”

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step

✓ **Exact packing.**

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 14 / 18 · PART 4 · CONCLUSION · DANIEL · TARGET 1 MIN

SINCE OUR LAST PRESENTATION

STEP 2 OF 5

- ✓ Exact packing.
- “Exact packing for small caches: done, it is the re-pack. Zoo is now proven optimal.”

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step
✓ **Annealing.**

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 14 / 18 · PART 4 · CONCLUSION · DANIEL · TARGET 1 MIN

SINCE OUR LAST PRESENTATION

STEP 3 OF 5

- ✓ Annealing.
- "Simulated annealing: done, and tested against LNS. LNS won, so we kept LNS."

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step

✓ **Faster code.**

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 14 / 18 · PART 4 · CONCLUSION · DANIEL · TARGET 1 MIN

SINCE OUR LAST PRESENTATION

STEP 4 OF 5

- ✓ Faster code.
- "Faster code: the C++ port, kittens in 13 seconds."

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step

✓ **Best strategy + now.**

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SINCE OUR LAST PRESENTATION

STEP 5 OF 5

- ✓ Best strategy + now.
- "Pick the best strategy: one command runs everything and keeps the best. Today we are second of twelve on the class benchmark, and third in the local contest."

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

15 · CONCLUSION

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

SCENE 15 / 18 · PART 4 · CONCLUSION · DANIEL · TARGET 1 MIN

CONCLUSION

STEP 1 OF 3

- The four layers and what each fixes.
- "So our solution is four layers. Each one fixes a weakness we measured in the one before, and none can make the answer worse."

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step

The one-sentence summary and three numbers.

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

CONCLUSION

STEP 2 OF 3

- The one-sentence summary and three numbers.
- "Build fast, improve with exact sub-problems, and solve exactly what is small enough. Third of twelve in the contest, second on the class benchmark, eight proven optima."

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next step

Questions. Backup scenes: 16 playground, 17 proofs, 18 complexity (type 1 6, 1 7, 1 8).

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

CONCLUSION

STEP 3 OF 3

- Questions. Backup scenes: 16 playground, 17 proofs, 18 complexity (type 16, 17, 18).
- "Thank you. We are happy to take your questions."

DANIEL · CONCLUSION

0:00

OF 2:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

16 · BACKUP · PLAYGROUND

Everyone takes over

BACKUP · PLAYGROUND

STEP 1 OF 1

- Playground: sliders change the instance; buttons show each layer's placement. Use it to answer "what if" questions.

EVERYONE · BACKUP FOR QUESTIONS

0:00

OF 10:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

17 · BACKUP · WHY EACH STEP IS SAFE

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

BACKUP · WHY EACH STEP IS SAFE

STEP 1 OF 1

- Proof sketches for the board: lazy heap, re-pack = knapsack, eviction, LNS acceptance, the ceiling, NP-hardness.

EVERYONE · BACKUP FOR QUESTIONS

0:00

OF 10:00

WHOLE TALK · TARGET 17:00

0:00

Next scene

18 · BACKUP · COST OF EACH STEP

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B

BACKUP · COST OF EACH STEP

STEP 1 OF 1

- Cost of each step and the size of the Google data sets.

EVERYONE · BACKUP FOR QUESTIONS

0:00

OF 10:00

WHOLE TALK · TARGET 17:00

0:00

Last step

◀ Back | Black | Next ▶

KEYS WORK HERE TOO: → ← B